UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

INGENIERÍA EN INFORMÁTICA Y SISTEMAS

BASES DE DATOS II — SEGUNDO CICLO 2026

DOCUMENTO COMPARATIVO DE PERSISTENCIA POLÍGLOTA

Entrega II: Plataforma de Historiales Clínicos Multimotor

Integrantes: - Ronaldo Emilio Méndez Mayorga — Carné: 1563224 - Brayan Benjamín Ruiz Quinilla — Carné: 1503724

Revisión técnica: 8 de noviembre de 2026
Repositorio: Rama `backend` — Arquitectura Hexagonal con FastAPI y React/Vite

---

1. Justificación de la Asignación de Motores por Institución

El enunciado del proyecto exige que cuatro instituciones médicas independientes compartan una misma plataforma de expedientes clínicos, operando cada una sobre un motor de base de datos distinto y justificado según sus necesidades operativas, volumen de datos y requerimientos de consistencia.

- **Hospital General — PostgreSQL (relacional).** Separa pacientes, citas, consultas, diagnósticos, tratamientos y recetas mediante claves foráneas. Cada petición de escritura usa una transacción; la cadena de varias peticiones no es una sola transacción.
- **Centro Médico Pediátrico — MySQL 8 (relacional).** Sus consultas, diagnósticos, controles de crecimiento y vacunas tienen tablas separadas y relaciones verificables, adecuadas para registros frecuentes y estructurados.
- **Laboratorio Clínico — MongoDB (documental).** Los parámetros de un examen varían; se almacenan dentro de la orden sin cambiar el esquema para cada prueba.

- **Clínica Odontológica — Firebase Firestore (documental).** Permite administrar pacientes, citas y atenciones sin operar un servidor de base de datos local. La conexión real requiere una cuenta de servicio válida.

---

2. Decisiones de Modelado por Institución

2.1 Hospital General (PostgreSQL — Esquema Relacional Normalizado)

- **Modelo relacional:** Se mantienen tablas separadas para `paciente`, `medico`, `especialidad`, `cita`, `consulta`, `catalogo_cie10`, `diagnostico`, `tratamiento`, `medicamento` y `receta`.
- **Integridad referencial:** Las relaciones clínicas usan claves foráneas. El adaptador exige un diagnóstico existente antes de registrar un tratamiento o prescribir.
- **Catálogos Normalizados:** Los diagnósticos enlazan directamente a la tabla `catalogo_cie10` mediante el estándar internacional de codificación clínica de la OMS.

2.2 Centro Médico Pediátrico (MySQL — Esquema Especializado)

- **Esquema operativo:** Además de `paciente` y `consulta_pediatrica`, el script `backend/scripts/mysql_pediatria.sql` crea tablas dedicadas a:
  - `control_crecimiento`: Monitorea peso (kg), talla (cm) y perímetro cefálico (cm) a lo largo del tiempo.
  - `vacuna_aplicada`: Registro de inmunizaciones con lote, dosis y fecha de aplicación.
  - Vínculo con tutores legales (`tutor_nombre`, `tutor_telefono`) integrado directamente en la entidad del paciente pediátrico.
  - `diagnostico`, `tratamiento_pediatrico`, `receta` y `cita_pediatrica`.

2.3 Laboratorio Clínico (MongoDB — Modelo Documental Híbrido)

- **Colección `pacientes`:** Almacena la ficha sociodemográfica del paciente de forma independiente (`_id`, `nombre`, `dpi`, `fecha_nacimiento`, `telefono`, `estado`).
- **Colección `ordenes`:** Cada orden de estudio clínico contiene:
  - Referencia al paciente (`id_paciente`).
  - Matriz embebida de `examenes[]`.
  - Matriz embebida interna de `resultados_parametros[]` (ej. parámetro, valor medido, rango de referencia, unidad).

- Matrices embebidas de `diagnosticos[]`, `tratamientos[]` y `recetas[]`.

2.4 Clínica Odontológica (Firebase Firestore — Colecciones y Subdocumentos)

- **Colección `/pacientes/{id}`:** Datos de filiación del paciente dental.
- **Colección `/citas/{id}`:** Citas agendadas con referencia a paciente y odontólogo.
- **Colección `/atenciones/{id}`:** Registros de atenciones con arrays embebidos de `procedimientos[]` (pieza y procedimiento), `diagnosticos[]`, `tratamientos[]` y `recetas[]`.

---

3. Decisión Crítica NoSQL: Embebido (Embedded) vs. Referenciado (Referenced)

Se referencian entidades con ciclos de vida distintos y se embeben datos que se consultan juntos.

3.1 Análisis en MongoDB (Laboratorio Clínico)

La colección `pacientes` contiene identidad y estado. Cada documento de `ordenes` guarda `id_paciente` como referencia y embebe un arreglo `examenes[]`. Cada examen embebe `resultados_parametros[]` con nombre, valor, unidad y rango de referencia.

- **Referencia paciente–orden:** Un paciente acumula órdenes sin límite conocido; separarlas evita que su ficha crezca hacia el límite de 16 MB de BSON. La identidad cambia con poca frecuencia y las órdenes se agregan continuamente.
- **Parámetros embebidos:** Se leen junto con su examen en una sola consulta y comparten su ciclo de vida; no requieren una colección independiente ni operaciones `$lookup`.

3.2 Análisis en Firestore (Clínica Odontológica)

Firestore mantiene `/pacientes/{id}` como documento raíz. Las colecciones `citas` y `atenciones` guardan referencias (`id_paciente`, `paciente_ref` e `id_cita`) en lugar de copiar la ficha completa del paciente. Así se evita duplicar identidad y una cita puede consultarse por paciente sin inflar un único documento de historial.

Cada documento de `atenciones` contiene arreglos embebidos acotados por encuentro: `procedimientos[]`, `diagnosticos[]`, `tratamientos[]` y `recetas[]`. Estos datos suelen leerse junto con la atención y se actualizan sobre ese mismo documento; el adaptador crea cita y atención en un batch. Las citas y atenciones se mantienen separadas porque tienen ciclos de vida distintos. La aplicación consulta colecciones por `id_paciente` y no dispone de joins; por eso esta decisión favorece lecturas de una atención, mientras evita agregar al documento del paciente arreglos que crecerían sin límite. El tamaño de los arreglos de una atención debe vigilarse si el sistema se amplía. La búsqueda parcial por nombre recorre la colección de pacientes y limita la respuesta a 25 coincidencias; para un volumen de producción se necesitaría un campo normalizado e indexado.

4. Garantía de Consistencia y Transacciones: SQL vs. NoSQL

Dimensión	Enfoque en SQL (PostgreSQL & MySQL)	Enfoque en NoSQL (MongoDB & Firestore)
Modelo de consistencia	Las tablas y claves foráneas protegen cada escritura.	Los datos embebidos de una orden o atención se actualizan de manera atómica dentro de su documento.
Transacciones implementadas	Cada petición abre una transacción y confirma o revierte su propia operación. Una secuencia de peticiones HTTP no constituye una única transacción.	MongoDB modifica una orden por operación. Firestore crea una cita atendida y su atención en un batch; los diagnósticos, tratamientos y recetas se agregan después mediante escrituras independientes.
Validación de referencias	El motor impone las claves foráneas.	Los adaptadores comprueban la existencia del paciente y de la orden o atención. No hay una transacción distribuida entre instituciones.

5. Seguridad y Control de Acceso

La seguridad implementada se estructura en tres capas:

1. **Autenticación Basada en JSON Web Tokens (JWT):**
   - Cada usuario inicia sesión a través de `POST /auth/login` con sus credenciales institucionales.
   - El token generado incluye una firma criptográfica con algoritmo `HS256`, un tiempo de expiración de 8 horas y los claims:

```
{
  "sub": "medico@hospital.gt",
  "institution": "hospital",
  "rol": "medico",
  "user_id": 1,
  "exp": 1790000000
}
```

2. **Permisos por rol y enrutamiento aislado:**
   - La capa de routing intercepta el claim `institution` en cada solicitud HTTP mediante la dependencia `Depends(get_current_institution)`.
   - Un usuario con token del `Hospital General` **no puede acceder ni escribir** en las colecciones de MongoDB del Laboratorio ni en la base de Pediatría. El enrutador conduce obligatoriamente su solicitud al adaptador de su motor asignado.
   - El backend comprueba permisos por operación. La cuenta de secretaría puede registrar pacientes y citas, pero no diagnósticos ni recetas; su respuesta de historial no incluye datos clínicos. El técnico de laboratorio puede registrar órdenes y resultados, pero no prescribir. Las cuentas clínicas realizan esas operaciones.
3. **Credenciales y Secretos Segregados:**
   - Cada motor cuenta con sus propias credenciales de conexión en variables de entorno segregadas en `.env` (puertos, usuarios, contraseñas y rutas de certificados protegidos en `.gitignore`).
   - El navegador no contiene contraseñas. Las cuentas locales de demostración usan hash PBKDF2; `JWT_SECRET` se configura en el entorno y, si falta, se genera una clave efímera.

- Estas cuentas fijas son para la demostración académica. Para información clínica real todavía haría falta un almacén administrable de usuarios, auditoría y revisión de privacidad.

---

6. Arquitectura Hexagonal y Principio de Sustitución

El cliente React envía JSON y JWT a los controladores FastAPI. Los controladores obtienen el repositorio del usuario autenticado mediante inyección de dependencias. El puerto `IRepository` define las operaciones clínicas comunes. Cada adaptador traduce esas operaciones a su motor:

1. `HospitalPostgresAdapter` → PostgreSQL.
2. `PediatriaMysqlAdapter` → MySQL.
3. `LaboratorioMongoAdapter` → MongoDB.
4. `OdontologiaFirebaseAdapter` → Firestore.

- **Independencia de la Lógica de Negocio:** Ningún controlador de FastAPI conoce sentencias SQL ni sintaxis BSON de MongoDB.
- **Operaciones compartidas y especializadas:** El puerto `IRepository` cubre el ciclo clínico común. Las rutas de resultados, crecimiento, vacunas y procedimientos invocan funciones propias del adaptador correspondiente, con la misma interfaz visual y autorización por institución.

---

7. Evidencia de Pruebas de Integración y Funcionamiento

El script `backend/tests/test_endpoints.py` crea expedientes sintéticos, valida registro, búsqueda por CUI y por nombre, consulta, diagnóstico, tratamiento, receta, historial y archivo. También comprueba contraseña inválida, respuestas 404 y bloqueo de escrituras después del archivo. Incluye los módulos propios de cada institución.

```
cd backend
source .venv/bin/activate
python tests/test_endpoints.py
```

Resultado de la re-verificación del 8 de noviembre de 2026

- **PostgreSQL / hospital:** 17/17 pruebas de API real, incluidas citas, tratamiento y búsqueda parcial por nombre.
- **MySQL / pediatría:** 19/19 pruebas de API real, incluidas citas, crecimiento, vacunas y búsqueda parcial por nombre.

- **MongoDB / laboratorio:** 16/16 pruebas de API real, incluidos parámetros medidos de resultados y búsqueda parcial por nombre.
- **Firestore / odontología:** 18/18 pruebas de API real, incluidas citas, procedimientos y archivo. Se verificó que la base `(default)` del proyecto corresponde a Firestore Standard en `nam5`; también pasó la búsqueda parcial por nombre.

Total de integración en vivo: 70/70 en los cuatro motores, tras iniciar el contenedor MongoDB. Las pruebas locales de adaptadores Firestore y MongoDB pasaron 3/3. También se cargaron y confirmaron tres expedientes sintéticos por institución con los scripts de precarga. La credencial de servicio permanece fuera de Git en `backend/serviceAccountKey.json`.